

Rajarata University of Sri Lanka
Department of Computing



ICT1402

PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 06

Variables and Data Types

K.A.S.H. Kulathilake

Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

- At the end of this lecture students should be able to;
 - Define Keywords / Reserve Words in C programming language.
 - Define Identifiers, Variable, Data Types, Constants and statements in C Programming language.
 - Justify the internal process with respect to the variable declaration and initialization.
 - Apply Variable Declaration and Variable initialization statement.
 - Assigning values to variables.
 - Apply taught concepts for problem solving.

Keywords / Reserve Words

- Keywords are **predefined, reserved words** used in programming that have **special meanings to the compiler**.
- As C is a case sensitive language, all keywords must be written in lowercase.

Keywords / Reserve Words (Cont...)

auto	break	case	char	const	continue
default	do	double	else	enum	extern
float	for	goto	if	inline	int
long	register	restrict	return	short	signed
sizeof	static	struct	switch	typedef	union
unsigned	void	volatile	while	_Alignas	_Alignof
_Atomic	_Bool	_Complex	_Generic	_Imaginary	_Noreturn
_Static_assert	_Thread_local				

Keywords / Reserve Words (Cont...)

- Data Types: These keywords are used to declare variable types or define the kind of data a variable can hold.

Keyword	Description
<code>int</code>	Integer data type (whole numbers)
<code>float</code>	Floating-point data type (single precision)
<code>double</code>	Floating-point data type (double precision)
<code>char</code>	Character data type (holds a single character)
<code>void</code>	Represents "nothing" or "no return type" for functions
<code>_Bool</code>	Boolean type (C99 and later) for <code>true</code> / <code>false</code> values
<code>_Complex</code>	Complex number data type (C99)
<code>_Imaginary</code>	Imaginary data type (C99)

Keywords / Reserve Words (Cont...)

- Control Flow Statements: These keywords control the execution flow of the program.

Keyword	Description
<code>if</code>	Conditional statement (if true, execute code)
<code>else</code>	Executes code if the <code>if</code> condition is false
<code>switch</code>	Selects one code block from many possible options
<code>case</code>	Defines a possible path in a <code>switch</code> statement
<code>default</code>	Default path in a <code>switch</code> statement if no case matches
<code>for</code>	Loop with a control variable (count-controlled loop)
<code>while</code>	Loop with a condition (condition-controlled loop)
<code>do</code>	Executes the loop at least once, then checks the condition
<code>goto</code>	Transfers control to a labeled statement (not recommended)

Keywords / Reserve Words (Cont...)

- Loop Control Statements: These keywords control how loops behave.

Keyword	Description
<code>break</code>	Terminates the loop or switch statement prematurely
<code>continue</code>	Skips the current iteration of a loop and moves to the next iteration
<code>return</code>	Exits from a function and optionally returns a value to the caller

Keywords / Reserve Words (Cont...)

- Storage Class Specifiers: These keywords define the scope, visibility, and lifetime of variables and functions.

Keyword	Description
<code>auto</code>	Default storage class for local variables (rarely used)
<code>static</code>	Variable retains its value across function calls (used for global/local scope)
<code>extern</code>	Declares a global variable or function defined in another file
<code>register</code>	Suggests to store the variable in a CPU register for faster access (optional)
<code>const</code>	Declares a read-only constant variable that cannot be changed after initialization
<code>volatile</code>	Tells the compiler that the value of a variable can change at any time (useful for memory-mapped registers)
<code>restrict</code>	Used in pointers to indicate that no other pointer will modify the object (C99)

Keywords / Reserve Words (Cont...)

- Type Qualifiers: These keywords define constraints on how a variable or pointer behaves.

Keyword	Description
<code>const</code>	Declares a variable as read-only (cannot be modified)
<code>volatile</code>	Informs the compiler not to optimize code involving this variable because its value can change unexpectedly
<code>restrict</code>	Declares that the pointer is the only reference to that memory address (C99)

Keywords / Reserve Words (Cont...)

- Type-Defining Keywords: These keywords define user-defined types or type aliases.

Keyword	Description
<code>typedef</code>	Defines a new name (alias) for an existing type
<code>struct</code>	Defines a structure, a collection of variables of different types
<code>union</code>	Defines a data structure where all members share the same memory space
<code>enum</code>	Defines an enumeration, a set of named integer constants

Keywords / Reserve Words (Cont...)

- Memory Management Keywords (Dynamic Allocation): These keywords are used for dynamic memory allocation in C.

Keyword	Description
<code>sizeof</code>	Returns the size in bytes of a variable, data type, or structure
<code>alignof</code>	Returns the alignment requirement of a type (introduced in C11)
<code>_Alignas</code>	Specifies the alignment of a variable (C11)

Note: While `malloc`, `free`, `calloc`, and `realloc` are often used for dynamic memory allocation, they are not keywords but functions from the `stdlib.h` header.

Keywords / Reserve Words (Cont...)

- Function Modifiers: These keywords affect function behavior.

Keyword	Description
<code>inline</code>	Suggests that a function should be inlined (its code is inserted directly into the calling location)
<code>_Noreturn</code>	Tells the compiler that a function does not return a value (C11)

- Special Purpose Keywords: These keywords have special usage in C, especially in multi-threaded programming (C11) or low-level programming.

Keyword	Description
<code>_Atomic</code>	Declares atomic objects (thread-safe objects)
<code>_Thread_local</code>	Declares thread-local storage (C11)
<code>_Static_assert</code>	Compile-time assertion (C11)

Identifiers

- An *identifier* in C consists of a sequence of letters, digits, or underscore characters.
- Identifiers are set when you declare **variables, arrays, enumerations, structures, union, typedef and functions**.
- Identifier **must be unique**.
- They are created to give unique name to an entity to identify it during the execution of the program.
- You can choose any name for an identifier (excluding keywords).
- However, if you give meaningful name to an identifier, it will be easy to understand and work on for you and your fellow programmers.

Identifiers (Cont...)

- **Rules for writing an identifier**
 - A valid identifier can have letters (both uppercase and lowercase letters), digits and underscores.
 - The first letter of an identifier should be either a letter or an underscore.
 - *However, it is discouraged to start an identifier name with an underscore.*
 - There is no rule on length of an identifier.
 - However, the first 31 characters of identifiers are discriminated by the compiler.

Variable

- Programming language enable you to assign symbolic names, known as *variable names*, for **storing program computations and results**.
- A variable name can be chosen by you in a **meaningful** way to reflect the type of value that is to be stored in that variable.
- Variables can be used to store integers, floating-point numbers, characters, strings and even pointers to locations inside the computer's memory.

Variable (Cont...)

- The rules for forming variable names:
 - They must begin with a letter or underscore (`_`) and can be followed by any combination of letters (upper- or lowercase), underscores, or the digits 0–9.
 - The following is a list of valid variable names.
 - `sum`
 - `pieceFlag`
 - `i`
 - `J5x7`
 - `Number_of_moves`
 - `_sysflag`

Variable (Cont...)

- Examples for invalid Variable Names
 - sum\$value - \$ is not a valid character.
 - piece flag - Embedded spaces are not permitted.
 - 3Spencer - Variable names cannot start with a number.
 - int - int is a reserved word.
- You should always remember that upper- and lowercase letters are distinct in C.
 - Therefore, the variable names sum, Sum, and SUM each refer to a different variable.

Variable (Cont...)

- It's typically not practical to use variable names that are too long—just because of all the extra typing you have to do.

- For example,

- although the following line is valid

```
theAmountOfMoneyWeMadeThisYear =  
theAmountOfMoneyLeftAttheEndOfTheYear –  
theAmountOfMoneyAtTheStartOfTheYear;
```

- this line

```
moneyMadeThisYear = moneyAtEnd – moneyAtStart;
```

conveys almost as much information in much less space.

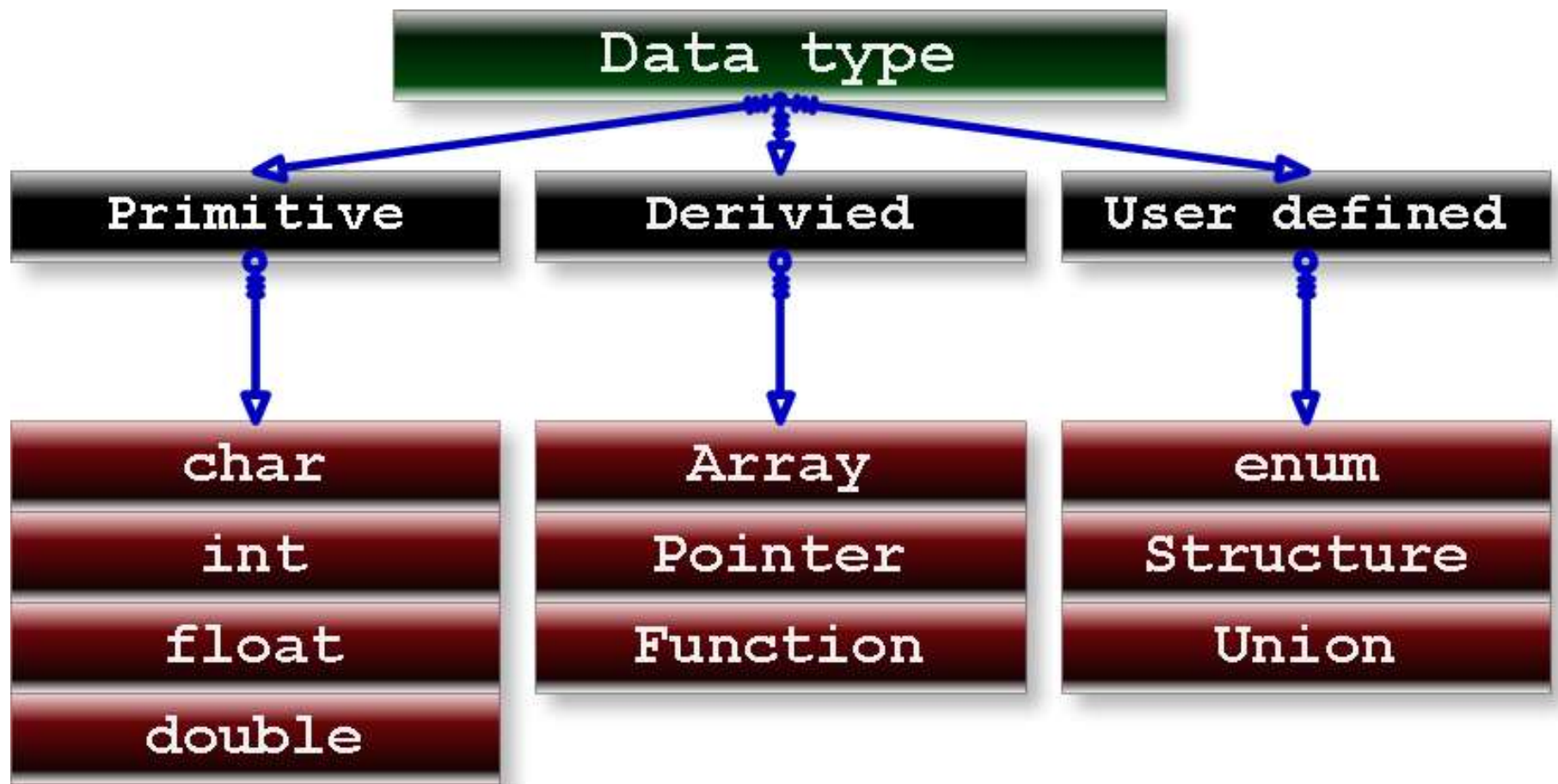
****** Always remember to pick names that reflect the intended use of the variable

Data Types

- Now we know that, variables are placeholders used to store values.
- The data type of a variable determines how the bits representing those values are stored in the computer's memory.
- When you declare a variable, you can also supply a data type for it.
- All variables have a data type that determines what kind of data they can store.

Data Types (Cont...)

Void is another data type used to indicate “no type”



Data Types (Cont...)

- The C programming language provides int, float and char as basic data types.
- Additionally, there are double and _Bool types available.
 - int – for storing integers.
 - float – for storing floating-point numbers.
 - double – for storing double precision floating point numbers.
 - char – for storing a single character.
 - _Bool - for indicating an on/off, yes/no, or true/false situation. For storing value 1 or 0.

Data Types (Cont...)

- **Storage size and Ranges**

- Every value, whether it's a character, integer, or floating-point number, has a range of values associated with it.
- This range has to do with the amount of storage that is allocated to store a particular type of data.
- In general, that amount is not defined in the language.
- It typically depends on the computer you're running, and is, therefore, called *implementation-* or *machine-dependent*.
- For example, an integer might take up 32 bits on your computer, or perhaps it might be stored in 64.

Data Types (Cont...)

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295

Data Types (Cont...)

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

Data Types & Constants

- In C, any number, single character, or character string is known as a *constant*.
 - E.g. the number 58 represents a constant integer value.
 - The character string "Programming in C is fun.\n" is an example of a constant character string.
- There are five different types of constants available in C programming language;
 - Integer constants
 - Floating point constants
 - Character constants
 - Character String Constants
 - Enumeration Constants

Data Types & Constants (Cont...)

- Integer Constants

- An integer constant is a sequence of digits, optionally preceded by a plus or minus sign.
 - E.g. -5 , 0, 100
- If the first digit is 0(zero), the integer is taken as an octal constant, in which case all digits that follow must be from 0 to 7.
 - E.g. 050
- If the first digit is 0 (zero) and is immediately followed by the letter x (or X), the integer is taken as a hexadecimal constant, and the digits that follow can be in the range from 0 to 9 or from a to f (or from A to F).
 - E.g. 0X50, 0xffff, 0xABF

Data Types & Constants (Cont...)

- The suffix letter l or L can be added to the end of a decimal integer constant to make it a long int constant.
- If the value can't fit into a long int, it's treated as a long long int.
- The suffix letters ll or LL can be added to the end of a decimal integer constant to make it a long long int.

long int constant → 100000l or 100001L
long long int constant → 100000ll or 100001LL

Data Types & Constants (Cont...)

- The suffix `u` or `U` can be added to the end of an integer constant to make it unsigned.
- If the constant is too large to fit inside an unsigned int, it's taken as an unsigned long int.
- If it's too large for an unsigned long int, it's taken as an unsigned long long int.

unsigned → `10u` or `100U`

Data Types & Constants (Cont...)

- Both an unsigned and a long suffix can be added to an integer constant to make it an unsigned long int.
- If the constant is too large to fit in an unsigned long int, it's taken as an unsigned long long int.
- Both an unsigned and a long long suffix can be added to an integer constant to make it an unsigned long long int.
- If an unsuffixed decimal integer constant is too large to fit into a signed int, it is treated as a long int.
- If it's too large to fit into a long int, it's treated as a long long int.
- In valid integers → 1000, -9945
- Print format character → %i, %o, %x

Data Types & Constants (Cont...)

- If an **unsuffixed octal or hexadecimal integer** constant is too large to fit into a signed int, it is treated as an **unsigned int**.
- If it's too large to fit into an unsigned int, it's treated as a **long int**, and if it's too large to fit into a long int, it's treated as an **unsigned long int**.
- If it's too large for an unsigned long int, it's taken as a **long long int**.
- Finally, if it's too large to fit into a long long int, the constant is treated as an **unsigned long long int**.

Data Types & Constants (Cont...)

- [Link to the Sample Program](#)

Data Types & Constants (Cont...)

- Floating Point Constants
 - A floating-point constant consists of a sequence of decimal digits, a decimal point, and another sequence of decimal digits.
 - A minus sign can precede the value to denote a negative value.
 - Either the sequence of digits before the decimal point or after the decimal point can be omitted, but not both.
 - E.g 3., 125.8, −.0001,

Data Types & Constants (Cont...)

- If the floating-point constant is immediately followed by the letter e (or E) and an optionally signed integer, the constant is expressed in scientific notation.
- This integer (the *exponent*) represents the power of 10 by which the value preceding the letter e (the *mantissa*) is multiplied (for example, $1.5e-2$ represents 1.5×10^{-2} or .015).
 - E.g. $1.7e4$, $1.5e-3$
- Print format character → %f, %e for scientific representation

Data Types & Constants (Cont...)

- Floating-point constants are treated as double precision values by the compiler.
- The suffix letter f or F can be added to specify a float constant instead of a double constant.
- The suffix letter l or L can be added to specify a long double constant.

Data Types & Constants (Cont...)

- [Link to the Sample Program](#)

Data Types & Constants (Cont...)



Type double is very similar to type float, but it is used whenever the range provided by a float variable is not sufficient.

Variables declared to be of type double can store roughly twice as many significant digits as can a variable of type float.

Most computers represent double values using 64 bits.

All floating-point constants are taken as double values by the C compiler.

To explicitly express a float constant, append either an **f or F** to the end of the number, as follows: **12.5f**

To display a double value, the format characters %f or %e which are the same format characters used to display a float value, can be used.

Data Types & Constants (Cont...)

- Character Constants
 - A character enclosed within single quotation marks is a character constant.
 - E.g. 'A', 'a', '0', '&', '3'
 - Print format character → %c
 - How the inclusion of more than one character inside the single quotation marks is handled is implementation-defined.
 - Special escape sequences are recognized and are introduced by the backslash character.

Data Types & Constants (Cont...)

▫ Escape sequence

Escape sequences are special character sequences that are used to represent certain characters that are difficult, impractical, or impossible to type directly into a source code. They are typically used to insert control characters, special symbols, or formatting into strings or character literals.

Escape sequences start with a backslash (\), followed by one or more characters that define the action or symbol to be produced.

Character	Meaning
<code>\a</code>	Audible alert
<code>\b</code>	Backspace
<code>\f</code>	Form feed
<code>\n</code>	Newline
<code>\r</code>	Carriage return
<code>\t</code>	Horizontal tab
<code>\v</code>	Vertical tab
<code>\\</code>	Backslash
<code>\"</code>	Double quote
<code>\'</code>	Single quote
<code>\?</code>	Question mark
<code>\octal</code>	Octal character value
<code>\universal</code>	Universal character name
<code>\Universal</code>	Universal character name
<code>\xhex</code>	Hexadecimal character value

Data Types & Constants (Cont...)

- [Link to the Sample Program](#)

Data Types & Constants (Cont...)

- Character String Constants
 - A sequence of zero or more characters enclosed within double quotation marks represents a character string constant.
 - Any valid character can be included in the string, including any of the escape characters listed previously.
 - The compiler automatically inserts a null character ('\0') at the end of the string.

Data Types & Constants (Cont...)



A `_Bool` variable is defined in the language to be large enough to store just the values 0 and 1.

The precise amount of memory that is used is unspecified.

`_Bool` variables are used in programs that need to indicate a Boolean condition.

When assigning a value to a `_Bool` variable, a value of 0 is stored as 0 inside the variable, whereas any nonzero value is stored as 1.

To make it easier to work with `_Bool` variables in your program, the standard header file `<stdbool.h>` defines the values `bool`, `true`, and `false`.

Data Types & Constants (Cont...)

Type	Constant Examples	printf chars
char	'a', '\n'	%c
_Bool	0, 1	%i, %u
short int	—	%hi, %hx, %ho
unsigned short int	—	%hu, %hx, %ho
int	12, -97, 0xFFE0, 0177	%i, %x, %o
unsigned int	12u, 100U, 0xFFu	%u, %x, %o
long int	12L, -200L, 0xffffL	%li, %lx, %lo
unsigned long int	12UL, 100uL, 0xffeeUL	%lu, %lx, %lo
long long int	0xe5e5e5e5LL, 50011	%lli, %llx, %llo
unsigned long long int	12u11, 0xffeeULL	%llu, %llx, %llo
float	12.34f, 3.1e-5f, 0x1.5p10, 0x1P-1	%f, %e, %g, %a
double	12.34, 3.1e-5, 0x.1p3	%f, %e, %g, %a
long double	12.34L, 3.1e-5L	%Lf, %Le, %Lg

Data Types (Cont...)

Type	Meaning
<code>int</code>	Integer value; that is, a value that contains no decimal point; guaranteed to contain at least 16 bits of precision.
<code>short int</code>	Integer value of reduced precision; takes half as much memory as an <code>int</code> on some machines; guaranteed to contain at least 16 bits of precision.
<code>long int</code>	Integer value of extended precision; guaranteed to contain at least 32 bits of precision.
<code>long long int</code>	Integer value of extraextended precision; guaranteed to contain at least 64 bits of precision.
<code>unsigned int</code>	Positive integer value; can store positive values up to twice as large as an <code>int</code> ; guaranteed to contain at least 16 bits of precision.
<code>float</code>	Floating-point value; that is, a value that can contain decimal places; guaranteed to contain at least six digits of precision.
<code>double</code>	Extended accuracy floating-point value; guaranteed to contain at least 10 digits of precision.
<code>long double</code>	Extraextended accuracy floating-point value; guaranteed to contain at least 10 digits of precision.

Data Types (Cont...)

<code>char</code>	Single character value; on some systems, sign extension might occur when used in an expression.
<code>unsigned char</code>	Same as <code>char</code> , except ensures that sign extension does not occur as a result of integral promotion.
<code>signed char</code>	Same as <code>char</code> , except ensures that sign extension does occur as a result of integral promotion.
<code>_Bool</code>	Boolean type; large enough to store the values 0 or 1.
<code>float _Complex</code>	Complex number.
<code>double _Complex</code>	Extended accuracy complex number.
<code>long double _Complex</code>	Extraextended accuracy complex number.
<code>void</code>	No type; used to ensure that a function that does not return a value is not used as if it does return one, or to explicitly "discard" the results of an expression. Also used as a generic pointer type (<code>void *</code>).

Constant Expression

- Expressions consisting entirely of constant values are called *constant expressions*.
 - E.g. Expression $128 + 7 - 17$ is a constant expression because each of the terms of the expression is a constant value.
 - $128 + 7 - i$ would not represent a constant expression because i was represented as a variable.

Statements

- A program statement is any valid expression (usually an assignment or function call) that is immediately followed by a semicolon, or it is one of the special statements like compound, statement, break, continue, do, for , go to, if, return, switch, while and null statement.

Variable Declaration

- When defining a particular variable, structure, union, enumerated data type, or typedef, the compiler does not automatically reserve any storage.
- The definition merely tells the compiler about the particular data type and (optionally) associates a name with it.
- After the definition has been made, variables can be declared to be of that particular data type.

Variable Declaration (Cont...)

- Declaration is done by simply listing the variables before the terminating semicolon of the definition.
 - E.g.
`data_type variable;`
`int age;`
- The language also enables storage to be allocated at the same time that a particular variable data type is defined.
- You can declare multiple variables in same type using a single statement as follows;
`int height, width;`

Assigning Values to Variables

- After declaring a variable with a type, you can add some content or value to that variable.
- This is known as assigning values to variable.
- Some times we have to initialize the variables to its initial or default values and it is based on the problem we are addressing.
- This is known as **variable initialization**.
- There are two ways to assign value to a variable:
 - Assign the value directly in the program.
 - Ask from user to input a value and then assign that value.

Assigning Values to Variables (Cont...)

NAME	VALUE	TYPE
number	123	int
sum	-456	int
pi	3.1416	double
average	-55.66	double

A variable has a name, stores a value of the declared type

Assigning Values to Variables (Cont...)

- Assign the value directly in the program.
 - Method 1: variable declaration and the assigning values as two separate statements;
`data_type variable;`
`variable = value;`
`int length;`
`length = 20;`
 - Here “=” is denoted as assignment operator.

Assigning Values to Variables (Cont...)

- Method 2: direct method for variable declaration and initialization;

```
data_type variable = value;  
int length = 20;
```

- Variable initialization;

```
int age = 0;  
float average = 0.0;  
int minimum_marks = 40;
```

Assigning Values to Variables (Cont...)

```
#include <stdio.h>
int main (void)
{
    int integerVar = 100;
    float floatingVar = 331.79;
    double doubleVar = 8.44e+11;
    char charVar = 'W';
    _Bool boolVar = 0;
    printf ("integerVar = %i\n", integerVar);
    printf ("floatingVar = %f\n", floatingVar);
    printf ("doubleVar = %e\n", doubleVar);
    printf ("doubleVar = %f\n", doubleVar);
    printf ("charVar = %c\n", charVar);
    printf ("boolVar = %i\n", boolVar);
    return 0;
}
```

Variable Declaration &
Initialization

Since the actual binary approximation of 331.79 might be something like 331.790008544921875, the first 6 decimal places are displayed as 331.790009 (because of rounding).

Assigning Values to Variables (Cont...)

```
#include <stdio.h>
int main (void)
{
    int sum;
    sum = 50 + 25;
    printf ("The sum of 50 and 25 is %i\n", sum);
    return 0;
}
```

declares the variable sum to be of type integer.

C requires that all program variables be declared before they are used in a program.

The number 50 is added to the number 25, and the result is stored (as indicated by the *assignment operator*, the equal sign) in the variable sum.

Assigning Values to Variables (Cont...)

```
printf ("The sum of 50 and 25 is %i\n", sum) ;
```

Formatting Character:

The character that immediately follows the percent sign specifies what *type* of value is to be displayed at that point. In the preceding program, the letter i is recognized by the printf routine as signifying that an integer value is to be displayed.

Whenever the printf routine finds the %i characters inside a character string, it automatically displays the value of the next argument to the printf routine.

Because sum is the next argument to printf, its value is automatically displayed after the characters “The sum of 50 and 25 is” are displayed.

Assigning Values to Variables (Cont...)

```
#include <stdio.h>
int main (void)
{
```

Variable Declaration

```
    int value1, value2, sum;
```

```
    value1 = 50;
```

```
    value2 = 25;
```

Variable Initialization

```
    sum = value1 + value2;
```

```
    printf ("The sum of %i and %i is %i\n",
            value1, value2, sum);
```

```
    return 0;
```

```
}
```


Assigning Values to Variables (Cont...)

- Ask from user to input a value and then assign that value.

```
#include <stdio.h>
int main()
{
    int number;
    printf("Type in a number : ");
    scanf("%i", &number);
    printf("The number you typed was %i\n", number);
    return 0;
}
```

Assigning Values to Variables (Cont...)

- **`scanf ("%i", &number);`**
 - The *scanf* routine, which accepts the response, has two arguments.
 - The first ("*%i*") specifies what type of data type is expected (ie char, int, or float).
 - The second argument (*&number*) specifies the variable into which the typed response will be placed.
 - In this case the response will be placed into the memory location associated with the variable *number*.
 - This explains the special significance of the *&* character (which means the address of).

Assigning Values to Variables (Cont...)

```
#include <stdio.h>

int main()
{
    int units;
    char letter;
    float unit_price = 12.5f;
    float amount = 0.0f;

    printf("Please enter # of units : ");
    scanf("%i", &units );

    printf("Please enter category id : ");
    scanf(" %c", &letter );

    amount = unit_price * units;

    printf("# of units = %i\n", units);
    printf("amount = %f - category = %c\n", amount, letter );

    return 0;
}
```

In the C programming language, the space in front of %c in the scanf statement (scanf(" %c", &letter);) is used to consume any whitespace characters, including newline characters or spaces, left in the input buffer from previous inputs. This is important to prevent issues with the input reading, especially when mixing scanf with other input functions.

Using Type Specifiers

```
#include <stdio.h>
int main (void)
{
    short shortvar = 34;
    printf ("int shortvar = %hi\n", shortvar);

    int variable = 1590;
    printf ("int variable = %i\n", variable);

    long int factorial = 1000001L;
    printf ("long int factorial = %li\n", factorial);

    long long int maxAllowedStorage = 100000011000LL;
    printf ("long long int maxAllowedStorage =
    %lli\n", maxAllowedStorage);
```

Using Type Specifiers (Cont...)

```
unsigned int counter = 10;  
printf ("unsigned int counter = %u\n", counter);
```

```
unsigned counter2 = 20;  
printf ("unsigned counter2 = %u\n", counter2);
```

```
unsigned char char_counter = 'a';  
printf ("unsigned char = %u\n", char_counter);
```

```
unsigned char_counter2 = 'A';  
printf ("unsigned char_counter2 = %u\n",  
char_counter2);
```

```
return 0;
```

```
}
```

Format Modifiers

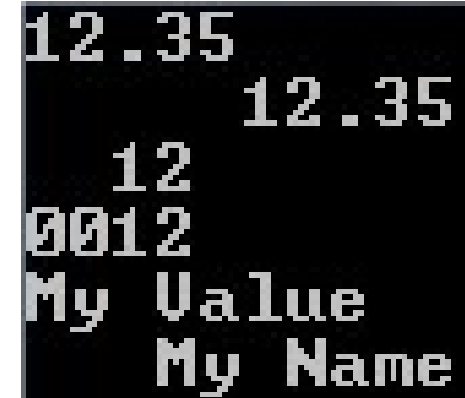
- Can be used to specify the required width of decimal integers and text strings.

Modifier	Description
%d , %i	Print as decimal integer
%6d	Print as decimal integer, at least six characters wide.
%f	Print as floating point
%6f	Print as floating point, at least six characters wide.
%.2f	Print as floating point, 2 characters after decimal point.
%6.2f	Print as floating point, at least 6 wide and 2 characters after decimal point.
%-4s	Print as four character string with left justified
%4s	Print as four character string with right justified.

Format Modifiers (Cont...)

```
#include <stdio.h>
int main()
{
    float value = 12.3456f;
    int number = 12;

    printf ( "%.2f\n", value );
    printf ( "%10.2f\n", value );
    printf ( "%4i\n", number );
    printf ( "%04d\n", number );
    printf ( "%-20s\n", "My Value" );
    printf ( "%10s\n", "My Name" );
    return 0;
}
```



```
12.35
          12.35
      12
0012
My Value
      My Name
```

Objective Re-cap

- Now you should be able to:
 - Define Keywords / Reserve Words in C programming language.
 - Define Identifiers, Variable, Data Types, Constants and statements in C Programming language.
 - Justify the internal process with respect to the variable declaration and initialization.
 - Apply Variable Declaration and Variable initialization statement.
 - Assigning values to variables.
 - Apply taught concepts for problem solving.

References

- Chapter 04, Appendix A - Programming in C, 3rd Edition, Stephen G. Kochan

Q & A

Next: Type Casting, Command Line Arguments and Defining Constants